Sistemes Distribuïts

Pràctica 2 Exclusió mútua distribuïda

Nom	Login
Antonio Duran	antonio.d
Arnau González	arnau.gonzalez

Contingut

Implementació i arquitectura 3
Organització del programa 3
Funcionament de l'algorisme 5
Arrencada i testimoni entre pesants 5
Lamport als processos A 5
Ricart Agrawala als processos B 5
Impressions i espera 5
Protocol i correcció 6
Rellotges i condicions d'entrada 6
Connexions entre processos 6
Proves i resultats 7
Interpretació de la sortida 7
Altres comprovacions efectuades 8
Rendiment i cost de comunicació 8
Cost de comunicació 8
Limitacions de la solució 9
Conclusions 10

Implementació i arquitectura

Hem organitzat el sistema en dos grups, cadascun amb un procés pesant i tres processos lleugers. Cada pesant crea els seus tres fills, que funcionen com a programes independents. El programa està implementat en C i tota la coordinació entre processos es fa amb sockets TCP.

L'objectiu principal d'aquesta estructura és evitar que dos processos escriguin a la pantalla alhora. Per aconseguir-ho, hem separat la coordinació en dos nivells: els pesants es passen un testimoni per decidir quin grup pot treballar, i els lleugers del grup actiu competeixen entre ells amb Lamport, en el cas dels A, o Ricart–Agrawala, en el dels B. El pesant dóna pas als tres fills alhora, de manera que no fixa un ordre A1, A2 i A3: aquest ordre el decideix l'algorisme d'exclusió mútua.

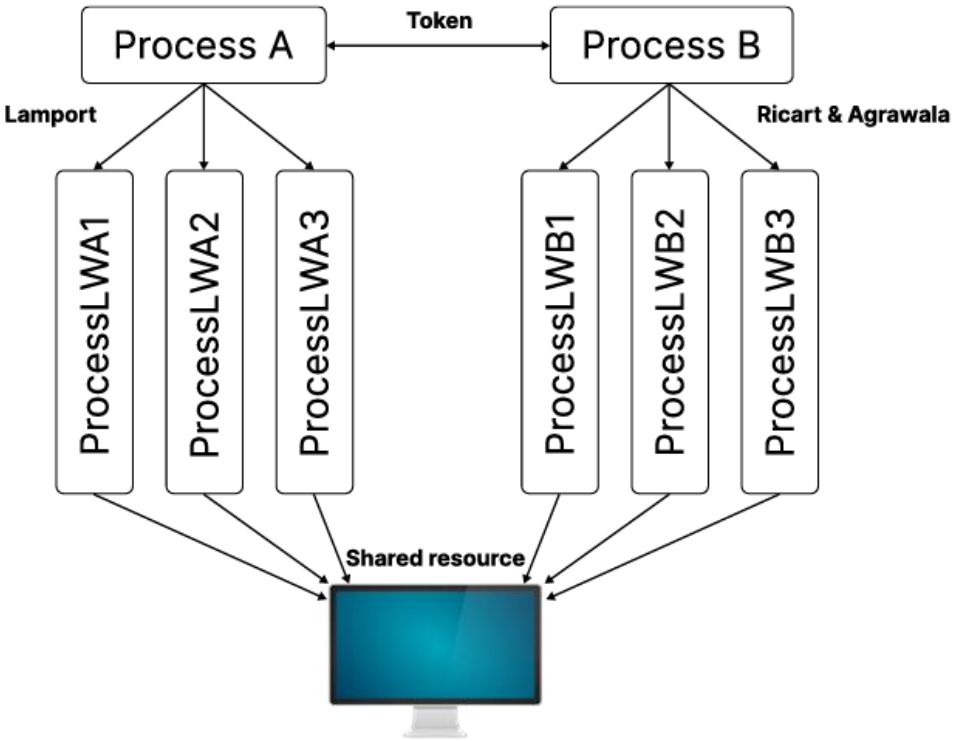


Figura 1. Els pesants es passen el testimoni i els lleugers comparteixen la pantalla.

Organització del programa

Fitxer	Responsabilitat
pesant.c	Creació dels fills, INICI, recollida dels FI i traspàs del testimoni.
lleuger_a.c	Peticions, confirmacions, cua i impressions del grup A.
lleuger_b.c	Peticions, confirmacions ajornades i impressions del grup B.
rellotges.c	Rellotge escalar, dependències directes i condició d'entrada de Lamport.
xarxa.c i comu.h	Connexions, trames, constants, estructures i declaracions comunes.

Hem utilitzat estructures de mida fixa perquè el sistema sempre té tres lleugers per grup. Així, cada procés guarda la informació dels seus dos companys i el seu propi estat sense necessitat de reservar memòria dinàmica.

Funcionament de l’algorisme

Arrencada i testimoni entre pesants

La sincronització comença amb una fase d'arrencada: cada pesant obre el seu servidor abans de crear els fills. Els lleugers es connecten al seu pesant i li envien IDENTITAT perquè pugui saber qui hi ha a cada connexió. A més dels seus tres fills, el pesant A accepta la connexió del pesant B. Durant aquesta fase també es preparen els canals entre els lleugers A.

Les connexions entre els lleugers B es preparen quan reben el primer permís d'inici, un cop el pesant ja ha acceptat tots els fills. B2 aprofita el canal d'escolta que havia utilitzat el seu pesant, de manera que no cal crear un punt d'entrada nou. A3 i B3 només inicien connexions i no necessiten servidor propi.

El pesant A comença amb l'únic testimoni. Quan un pesant té el torn, envia INICI als tres fills i espera un FI de cadascun abans de passar TESTIMONI a l'altre pesant. Aquest cicle es repeteix indefinidament. Recollir els FI per identificador no imposa l'ordre d'impressió: les respostes que arriben abans queden pendents al socket.

Lamport als processos A

En el grup A hem implementat l'algorisme de Lamport amb una cua que guarda una marca de temps per procés. Quan un lleuger vol escriure, registra la seva petició i envia REQUEST als dos companys amb la mateixa marca temporal. Com que cada procés només pot tenir una petició activa, n'hi ha prou amb reservar un espai per a cadascun i indicar si té alguna petició pendent.

En rebre REQUEST, el procés actualitza el rellotge, guarda la petició i respon ACK. Per entrar, necessita les dues confirmacions i que la petició pròpia tingui prioritat. Primer es compara la marca de temps i, si coincideix, té preferència l'identificador més petit. La decisió té en compte les confirmacions rebudes, les peticions pendents i les marques conegudes dels companys.

Finalment, quan el lleuger acaba les deu impressions, elimina la seva petició de la cua i envia RELEASE als companys perquè també la retirin. Després avisa el pesant amb FI. Així, el següent procés pot entrar quan compleixi les mateixes condicions.

Ricart Agrawala als processos B

En el grup B, la diferència principal és que el permís es decideix en el moment de respondre una petició. Si un lleuger no està competint, respon ACK directament. Si també vol entrar, compara les dues peticions: respon a la que té prioritat i deixa les altres pendents. Quan rep els dos ACK dels companys, pot començar a escriure. En acabar, respon les peticions ajornades i envia FI al pesant. D'aquesta manera, Ricart–Agrawala evita el missatge RELEASE de Lamport. Cada nova petició necessita les seves pròpies confirmacions.

Impressions i espera

Quan un lleuger aconsegueix el torn, mostra el seu identificador deu vegades seguides i fa una pausa d'un segon després de cada línia, també després de l'última. Durant aquest bloc conserva l'accés exclusiu a la pantalla i deixa pendents els missatges que li arriben. Un cop acaba, torna a atendre les comunicacions amb els altres processos. D'aquesta manera, les impressions de dos lleugers no es barregen.

Protocol i correcció

Hem utilitzat un protocol binari amb tres camps: el tipus de missatge, el procés d'origen i la marca de temps. Tots els missatges tenen una mida fixa i les dades es transmeten amb un ordre de bytes comú. Com que TCP pot lliurar només una part del missatge en cada lectura, el receptor espera fins a tenir-lo sencer abans d'interpretar-lo. També es comprova si la connexió s'ha tancat i es reprenen les operacions interrompudes.

Codi	Trama	Participants	Funció
1	IDENTITAT	Arrencada	Associar una connexió amb el procés que la inicia.
2	INICI	Pesant → fills	Autoritzar els tres lleugers a començar una ronda.
3	FI	Fill → pesant	Confirmar que ha acabat el seu bloc i ha alliberat l'accés.
4	TESTIMONI	Entre pesants	Cedir el torn a l'altre grup.
5	REQUEST	Entre companys	Demanar accés amb la marca de la petició.
6	ACK	Entre companys	Confirmar la petició; en B es pot ajornar.
7	RELEASE	Només entre A	Eliminar de la cua la petició del remitent.

Rellotges i condicions d'entrada

Per ordenar les peticions sense dependre de l'hora de l'ordinador, hem utilitzat rellotges lògics. L'escalar avança amb cada increment i, en rebre una marca, calcula el màxim entre el valor propi i el rebut més un. Els B utilitzen aquest rellotge. Els A utilitzen un rellotge directe per a Lamport i un escalar separat per coordinar-se amb el pesant.

El rellotge directe manté la marca pròpia i les últimes marques conegudes dels companys. Quan arriba un missatge, s'actualitza la informació del remitent i s'avança el rellotge propi tenint en compte el temps rebut. Només es transmet una marca temporal en cada missatge. A més, la marca original d'una petició es conserva mentre espera el torn, perquè és la que determina la seva prioritat.

Connexions entre processos

Hem mantingut un únic canal bidireccional entre cada parella de lleugers del mateix grup. Per evitar connexions duplicades, el procés amb identificador més gran inicia la connexió amb el més petit. Quan es connecten, els participants s'identifiquen perquè el receptor pugui associar cada canal amb el procés corresponent.

Proves i resultats

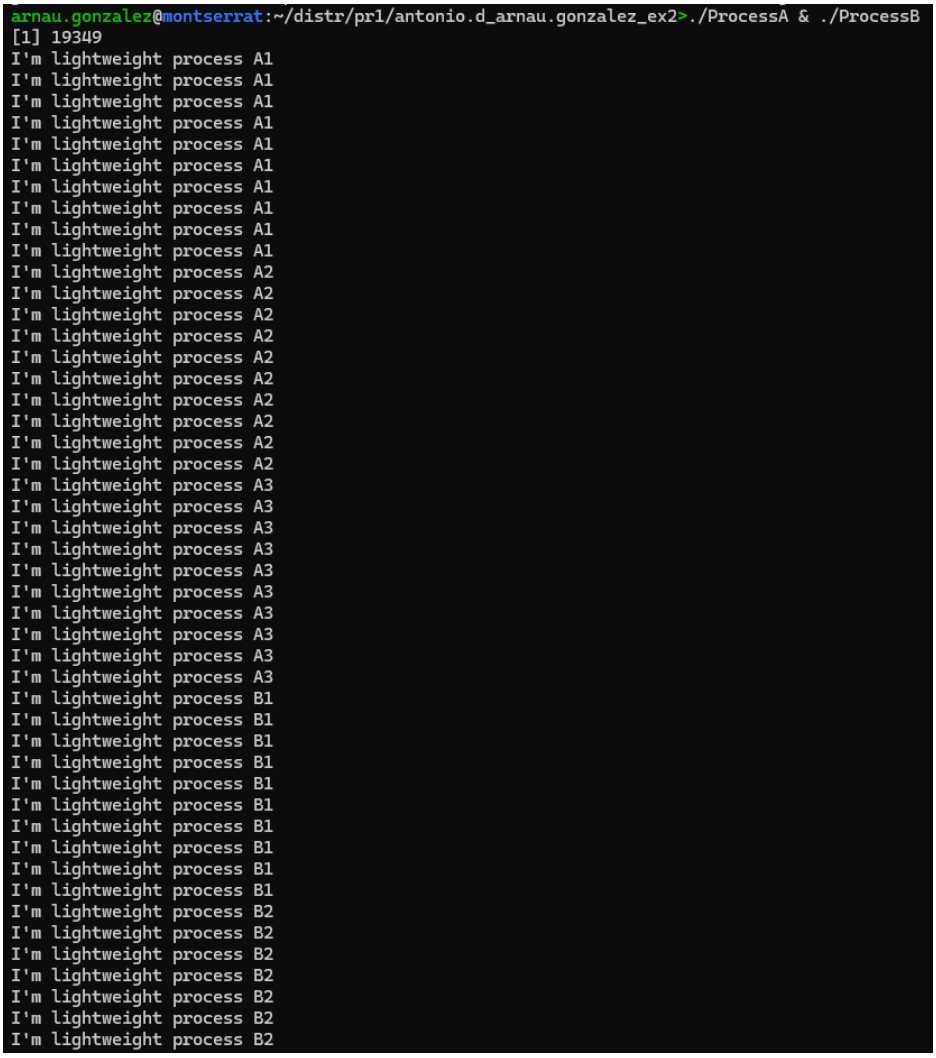
Per comprovar el funcionament del conjunt, hem iniciat els dos pesants i hem deixat que cadascun creés els seus tres fills. Hem repetit la prova canviant l'ordre d'arrencada dels pesants. La taula següent resumeix els dos escenaris i els resultats obtinguts:

Arrencada	Línies observades	Pausa observada	Comprovació
A abans que B	121	Aprox. 1 s	Dos cicles A/B complets i inici del següent torn.
B abans que A	61	Aprox. 1 s	Un cicle A/B complet i inici del següent torn.

En tots dos casos hem comprovat que cada lleuger mostra deu línies seguides, sense barrejar-les amb les d'un altre procés. També hem verificat que cada grup completa trenta línies amb els seus tres identificadors abans de donar pas a l'altre. Hem comprovat que entre dues impressions consecutives passa aproximadament un segon.

Interpretació de la sortida

La captura mostra com s'inicia l'execució i com els lleugers s'alternen per escriure a la pantalla.



En aquesta execució, A1, A2 i A3 mostren el seu identificador deu vegades cadascun. Les línies de cada procés apareixen juntes perquè només un lleuger té accés a la pantalla en cada moment. Quan els tres lleugers del grup A han acabat les seves impressions, comença el torn del grup B. A la captura es veu el bloc de B1 i l'inici del de B2.

Altres comprovacions efectuades

A més de les proves d'execució, hem comprovat per separat les actualitzacions dels rellotges, les confirmacions necessàries per entrar i els desempats de Lamport. També hem enviat una trama byte a byte per verificar que es rep sencera, hem tancat una connexió a mitja trama per comprovar el retorn d'error i hem verificat les conversions d'ordre de xarxa.

Finalment, en totes dues execucions hem provocat la desconnexió d'un fill del grup actiu. Els pesants han acabat i han recollit els sis fills. Aquesta prova comprova la neteja en aquest cas d'error; el programa no reinicia els processos ni recupera automàticament el testimoni.

Rendiment i cost de comunicació

Les pauses d'un segon tenen un pes important en el temps de funcionament. Cada entrada a la secció crítica conté deu esperes i els processos escriuen d'un en un. Per tant, un torn complet de tres lleugers acumula trenta esperes i un cicle dels grups A i B n'acumula seixanta. Aquests valors es dedueixen del bucle d'impressions; no són una mesura del temps total del programa, que continua executant-se indefinidament.

Cost de comunicació

Per comparar els dos algorismes, hem comptat els missatges que envia el codi en una ronda sense errors. La taula següent no inclou els missatges IDENTITAT de l'arrencada i recull el recompte del codi, no una captura de trànsit de xarxa:

Part del protocol	Per entrada d'un lleuger	Per torn del grup
Lamport als A	2 REQUEST + 2 ACK + 2 RELEASE = 6	3 × 6 = 18
Ricart–Agrawala als B	2 REQUEST + 2 ACK = 4	3 × 4 = 12
Coordinació del pesant	—	3 INICI + 3 FI + 1 TESTIMONI = 7

La diferència entre els dos grups és que Lamport necessita dos REQUEST, dos ACK i dos RELEASE per entrada, mentre que Ricart–Agrawala només necessita els dos REQUEST i els dos ACK. En un torn de tres lleugers, Ricart–Agrawala necessita 12 missatges entre companys i Lamport en necessita 18. La diferència correspon als sis missatges RELEASE. Si també comptem els missatges dels pesants, un cicle A/B suma 18 + 12 + 7 + 7 = 44 missatges. Aquesta reducció de trànsit no elimina les pauses: tots dos grups continuen fent trenta impressions amb les seves esperes.

Pel que fa a l'escalabilitat, la implementació està preparada per a tres lleugers per grup. No hem fet proves amb més participants, de manera que els resultats es limiten a aquesta composició del sistema.

Limitacions de la solució

La composició del sistema és fixa des de l'inici. La informació de les connexions, les peticions i les confirmacions està preparada per a tres lleugers per grup. A més, un cop establertes les connexions, es deixa d'acceptar nous participants. Per tant, no es poden afegir processos mentre el programa està en marxa.

Durant l'arrencada, si una connexió falla, es torna a intentar fins a deu vegades amb una pausa d'un segon. Això deixa un marge perquè l'altre procés estigui preparat. En canvi, l'espera de noves connexions no té temps límit: si un lleuger no arriba a iniciar-se, un procés pot quedar esperant una connexió que no arribarà.

La lectura completa dels missatges també té una limitació: si un emissor envia només una part de la trama i continua connectat sense enviar la resta, el receptor es pot quedar bloquejat. La prova de tancament a mitja trama comprova que es detecta una desconnexió, però no cobreix aquest cas d'un procés que es queda aturat.

D'altra banda, mentre un lleuger imprimeix no atén els sockets. Si falla un fill del grup que està esperant, el pesant pot detectar-ho al torn següent, ja que mentrestant espera el testimoni. El sistema tampoc pot continuar amb un participant menys: per passar el torn necessita els tres FI. Les prioritats dels rellotges decideixen l'ordre dins de cada grup, però no estableixen un ordre temporal conjunt entre els sis lleugers.

Quan es detecta un error, els processos tanquen les connexions obertes i els pesants aturen els fills que han creat i n'esperen la finalització. Aquesta neteja evita deixar fills en execució en aquests casos, però no permet reprendre la feina ni substituir un procés caigut. Tampoc hi ha cap mecanisme per regenerar el testimoni o gestionar que els rellotges superin el màxim valor representable.

Conclusions

Hem après a aplicar l'exclusió mútua que hem vist a classe a un recurs concret, la pantalla. El testimoni ens ha servit per separar els torns dels dos grups, mentre que els lleugers de cada grup s'han hagut de posar d'acord per escriure d'un en un. A la sortida hem vist com es combinen aquests dos mecanismes. Cada lleuger mostra deu línies seguides i el grup següent espera fins que han acabat els tres del grup actiu.

També hem après a utilitzar les marques dels rellotges lògics per decidir quina petició té prioritat. Si dues peticions tenen la mateixa marca, l'identificador del procés permet desfer l'empat. En aplicar Lamport i Ricart–Agrawala al mateix problema, hem vist que amb Lamport cal avisar els companys quan s'allibera el torn, mentre que amb Ricart–Agrawala es respon en aquell moment a les peticions que havien quedat pendents.

Finalment, hem posat en pràctica la comunicació entre processos amb sockets. Els missatges serveixen per demanar permís, confirmar peticions i avisar que s'ha acabat el torn. Una de les proves ens ha permès veure que un missatge pot arribar en diversos fragments i que cal reunir-los abans d'interpretar-lo.